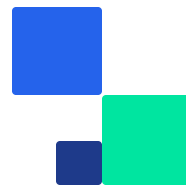




TS ACADEMY

File Handling & NumPy Arrays

Learn to read, write, and manage files in Python — then
supercharge your data work with NumPy.



● PART 1 · FILE HANDLING

What Is File Handling?

File handling gives your program the ability to work with files on your computer — just like how you use Notepad or Word to open, read, and write documents.



CSV Files

Datasets



JSON Files

APIs & Web



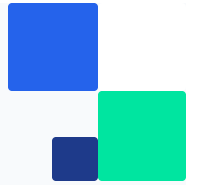
Text Files

Logs & Reports



Model Output

Results & Exports



● ANALOGY

Think of It Like a Diary



Open It

Access the file to read or write

1



Read or Write

Process the data inside

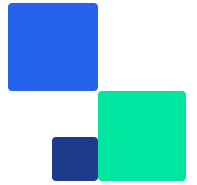
2



Close It

Release so others can use it

3



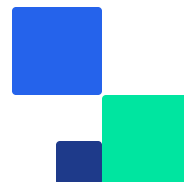
● BASICS

Opening & Closing Files

Use Python's built-in `open()` function:

```
# Opening a file file = open('data.txt', 'r') # 'r' = read mode content = file.read() # Read the entire file print(content) file.close() # Always close!
```

PYTHON



● FILE MODES

Three Essential Modes

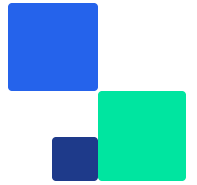
MODE	MEANING	BEHAVIOR
"r"	Read	Opens for reading — file must exist
"w"	Write	Creates or <i>overwrites</i> the file
"a"	Append	Adds to end — keeps existing content

```
file = open("data.txt", "w") file.write("Hello World") file.close()
```

WRITE

```
file = open("data.txt", "a") file.write("\nNew line") file.close()
```

APPEND



● BEST PRACTICE

The `with` Statement

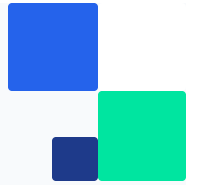
A context manager that **automatically closes** the file — even if errors occur.

✗ MANUAL WAY

```
file = open('data.txt', 'r') print(file.read()) file.close() # Easy to forget!
```

✓ THE SMART WAY

```
with open('data.txt', 'r') as file: print(file.read()) # Auto-closed here!
```



● WARNING

What Happens If You Don't Close?



Memory Leaks

Wasted memory your program never releases



File Locks

Other programs can't access the file



System Limits

OS limits how many files can be open



Corruption Risk

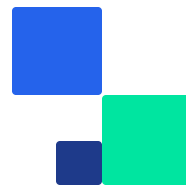
Data might not be fully written to disk



● PART 2

Introduction to NumPy Arrays

The backbone of data science in Python — fast, efficient,
and the foundation for Pandas, Scikit-learn, and TensorFlow.



• WHY NUMPY?

Lists vs. NumPy

✗ PYTHON LISTS

```
numbers = [1, 2, 3, 4] new = [] for n in numbers: new.append(n + 5) print(new) # [6, 7, 8, 9]
```

✓ NUMPY ARRAYS

```
import numpy as np arr = np.array([1, 2, 3, 4]) print(arr + 5) # [6 7 8 9]
```

50–100× faster Less memory No loops needed



• BENEFITS

Why Data Scientists Love NumPy



Speed

50-100x faster than Python lists



Memory

Uses significantly less memory



Vectorization

Operate on entire arrays at once



Functions

Thousands of math operations



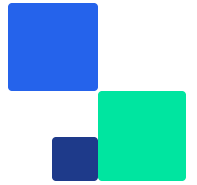
Foundation

Powers Pandas, Scikit-learn, TF



Real-World

Images, finance, genomics



• CREATING ARRAYS

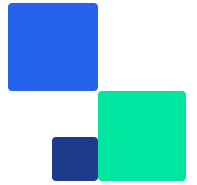
1D & 2D Arrays

1D Array — A single line of values

```
import numpy as np arr = np.array([10, 20, 30, 40]) print(arr) # [10 20 30 40]
```

2D Array — Rows & columns (matrix)

```
arr_2d = np.array([ [1, 2, 3], [4, 5, 6] ]) print(arr_2d) # [[1 2 3] # [4 5 6]]
```



• ATTRIBUTES

Array Properties

Every NumPy array carries metadata you can inspect:

```
arr = np.array([[1, 2, 3], [4, 5, 6]]) print(arr.shape) # (2, 3) → 2 rows, 3 columns  
print(arr.ndim) # 2 → two dimensions print(arr.dtype) # int64 → data type
```

PYTHON

.shape

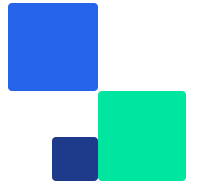
Rows × Columns

.ndim

Number of dimensions

.dtype

Data type (int, float...)



● OPERATIONS

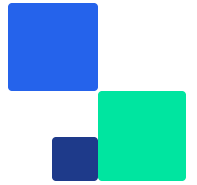
Math & Reshaping — No Loops

Vectorized Math

```
arr = np.array([1, 2, 3, 4]) print(arr + 10) # [11 12 13 14] print(arr * 2) # [ 2 4 6 8]
print(arr ** 2) # [ 1 4 9 16]
```

Reshaping

```
arr = np.array([1,2,3,4,5,6]) reshaped = arr.reshape(2, 3) print(reshaped) # [[1 2 3] # [4 5
6]]
```



● INDEXING & SLICING

Accessing Elements

1D Indexing

```
arr = np.array([10,20,30,40,50]) print(arr[0]) # 10 (first) print(arr[-1]) # 50 (last)  
print(arr[1:4]) # [20 30 40]
```

2D Indexing

```
m = np.array([[1,2,3], [4,5,6]]) print(m[0, 1]) # 2 (row 0, col 1) print(m[1, 2]) # 6 (row 1,  
col 2)
```

Filtering

```
arr = np.array([1, 2, 3, 4, 5]) print(arr[arr > 2]) # [3 4 5] – Boolean mask filtering
```



WHAT'S NEXT

Keep Learning

Video Tutorials

- Dave Gray — Python File Handling
- NumPy Tutorial Playlist — Data Science Entry

Documentation

- docs.python.org — Official Python Docs
- w3schools.com/python — Tutorials & Examples

Practice with the Jupyter notebook & ask questions in the group!